

CUDA Qualifiers

CPU
Host

`__global__`

`__device__`

`__host__`

`__shared__`

`__constant__`

`__managed__`

GPU
Device

- Functions: where code executes
- Variables: where data lives
- Built-ins: how threads find their work

Learning outcomes

1

Classify qualifiers
Identify function qualifiers
and memory-space qualifiers.

2

Read CUDA code
Explain which lines execute on
CPU and which execute on
GPU.

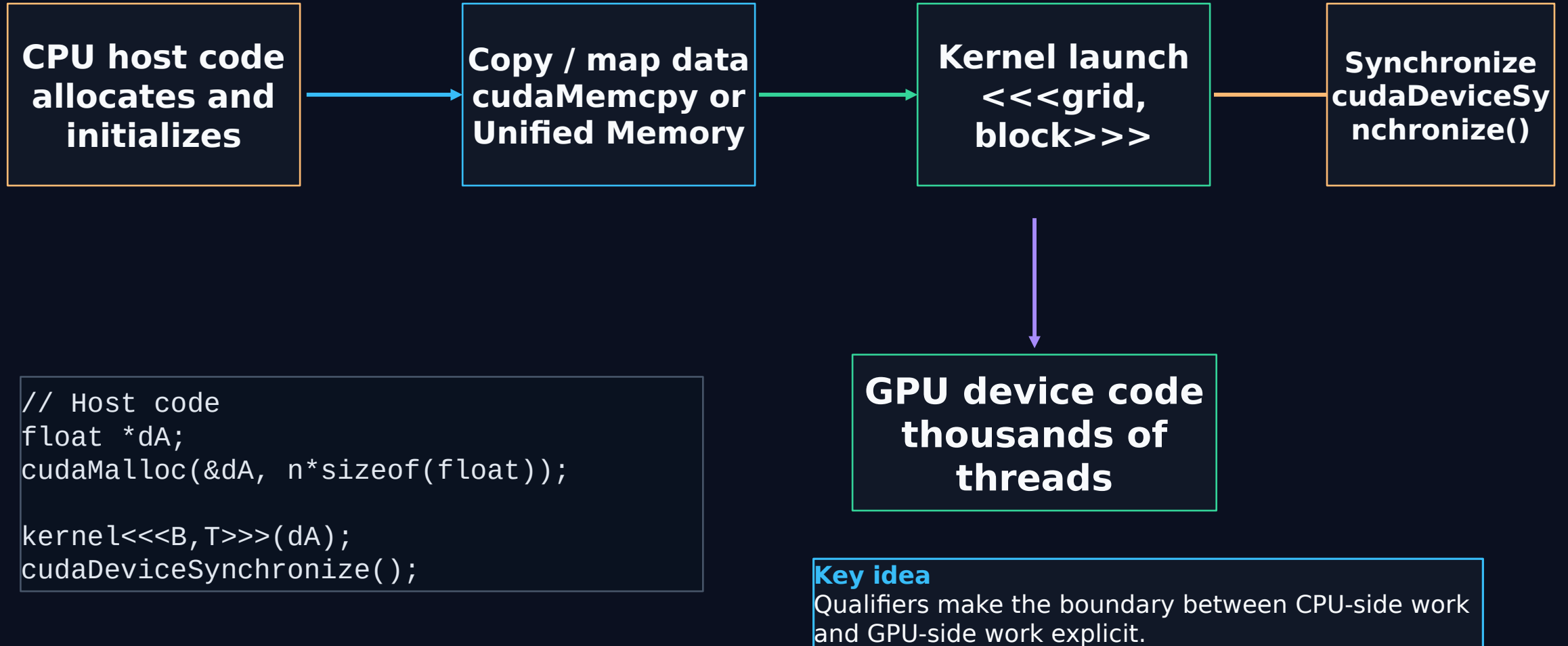
3

Choose memory space
Select `__shared__`, `__constant__`,
`__managed__`, or `__device__` for
common cases.

4

Avoid common bugs
Recognize illegal calls, missing
synchronization, and out-of-
bounds indexing.

CUDA execution flow



Mental model for every declaration

Question 1
Where does this function execute?

FUNCTION DECLARATIONS

```
__global__ void kernel(...);  
__device__ int helper(int x);  
__host__ void cpuFunc(...);  
__host__ __device__ float f(float x);
```

Question 2
Where is this variable stored?

VARIABLE DECLARATIONS

```
__device__ int g;  
__managed__ int u;  
__constant__ float table[64];  
__shared__ float tile[256];
```

__device__ is overloaded: for a function it means GPU-callable; for a variable it means global device memory.

Function qualifiers at a glance

Qualifier	Called from	Executes on	Main use
<code>__host__</code>	CPU code	CPU	Normal C/C++ function
<code>__device__</code>	GPU code	GPU	Helper function inside kernels
<code>__global__</code>	Usually CPU code	GPU	Kernel entry point
<code>__host__ __device__</code>	CPU or GPU code	CPU or GPU	Reusable utility function

Remember

`__global__` is the only ordinary kernel qualifier and the launch syntax uses triple angle brackets.

__global__ kernel functions

```
__global__ void vectorAdd(const float *A,
                        const float *B,
                        float *C, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n)
        C[i] = A[i] + B[i];
}

// host-side launch
vectorAdd<<<numBlocks, threadsPerBlock>>>(A, B, C, n);
```

Must return void

Runs once per GPU thread

**Launched with <<<grid,
block>>>**

Launch is asynchronous

Built-in variables for indexing



```
int i = blockIdx.x * blockDim.x +  
threadIdx.x;  
  
if (i < n) {  
    C[i] = A[i] + B[i];  
}
```

Example
blockIdx.x = 2
blockDim.x = 256
threadIdx.x = 17

i = 529

Why it matters

Without unique indexing, many threads may overwrite the same output location.

__device__ helper functions

```
__device__ float square(float x)
{
    return x * x;
}

__global__ void transform(float *a)
{
    int i = threadIdx.x;
    a[i] = square(a[i]);
}
```

**__global__
transform()**

calls

**__device__
square()**

- Used to modularize GPU-side logic.
- Cannot be called directly from ordinary CPU code.
- Often inlined by the compiler for performance.

__host__ and __host__ __device__

```
__host__ void initialize(float *h)
{
    // CPU-only function
}

__host__ __device__
float clamp01(float x)
{
    return x < 0 ? 0 : (x > 1 ? 1 : x);
}
```

**Plain C/C++ functions are
__host__ by default.**

**__host__ __device__ creates
two compiled versions.**

Caution

A dual-side function cannot freely use CPU-only libraries or GPU-only built-ins unless guarded.

Good use

Small math utilities, bounds checks, operators, and simple data-structure methods.

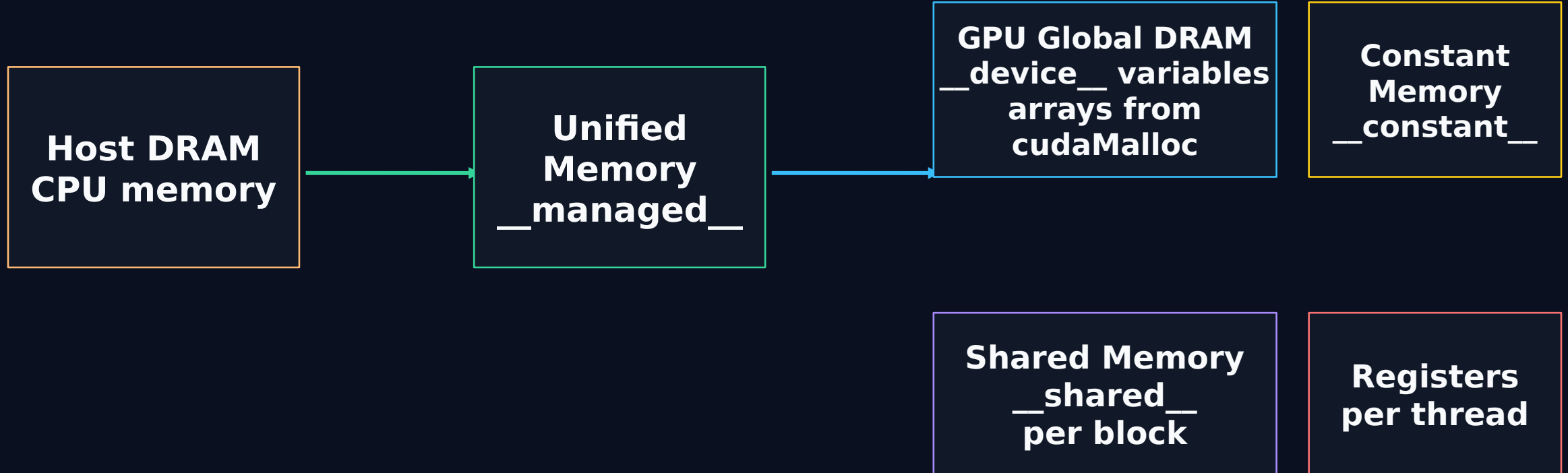
Memory-space qualifiers at a glance

Qualifier	Memory space	Visible to	Typical use
<code>__device__</code>	GPU global memory	All GPU threads	Global variables, counters, flags
<code>__managed__</code>	Unified Memory	CPU and GPU	Simple shared data model
<code>__constant__</code>	Constant memory	All GPU threads read	Coefficients, parameters, small tables
<code>__shared__</code>	On-chip shared memory	Threads in one block	Tiling, reductions, cooperation

Practical selection rule

Use the narrowest memory scope that supports the algorithm; narrower scope often improves locality and clarity.

Memory placement view



Speed intuition

On-chip memories are fast but limited. Global memory is large but slower. Unified Memory is convenient but access pattern still matters.

__device__ variables

```
__device__ int counter = 0;

__global__ void countKernel()
{
    atomicAdd(&counter, 1);
}

int h_counter;
cudaMemcpyFromSymbol(&h_counter,
                    counter,
                    sizeof(int));
```

Stored in GPU global memory

Accessible by device code

Host uses symbol-copy APIs

Concurrency note

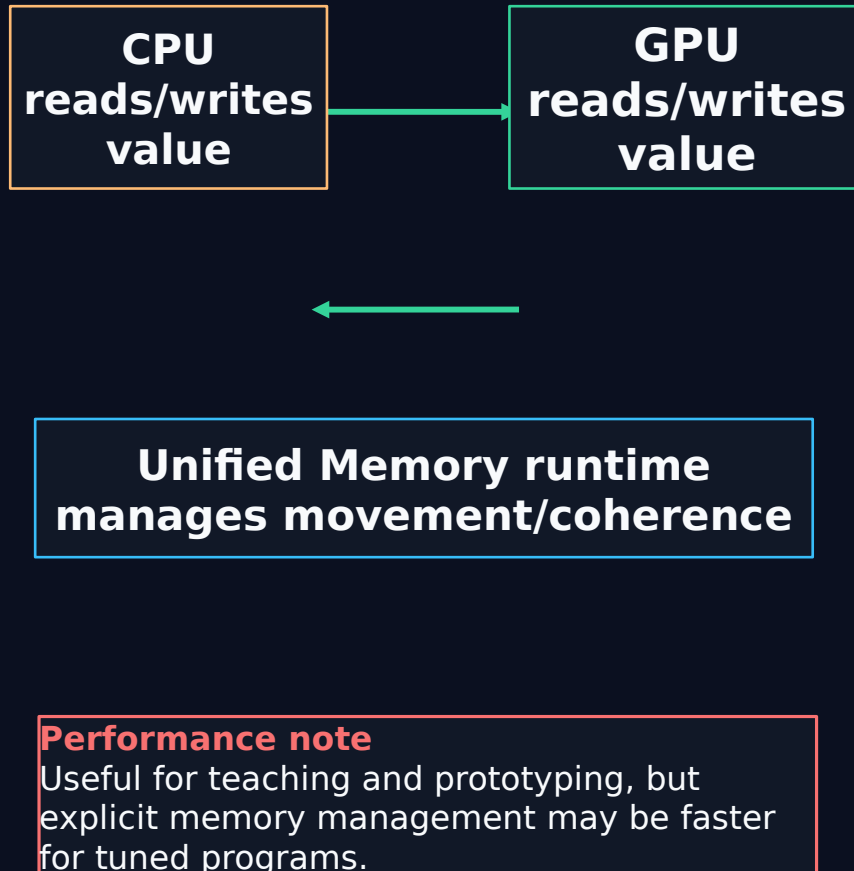
Multiple GPU threads writing the same variable require atomics or another synchronization design.

__managed__ variables

```
__managed__ int value;

__global__ void increment()
{
    value++;
}

int main()
{
    value = 10;
    increment<<<1,1>>>();
    cudaDeviceSynchronize();
    printf("%d", value);
}
```



__constant__ memory

```
__constant__ float coeff[256];

// Host initializes once
cudaMemcpyToSymbol(coeff, h_coeff,
                    256*sizeof(float));

__global__ void filter(float *out)
{
    int i = threadIdx.x;
    out[i] = coeff[i];
}
```

Read-only inside kernels

**Host initializes with
cudaMemcpyToSymbol()**

**Efficient when many
threads read the same
address**

Examples

Filter coefficients, scalar parameters, small lookup tables, and read-only configuration data.

__shared__ memory

```
__global__ void reduce(float *x)
{
    __shared__ float tile[256];

    int t = threadIdx.x;
    tile[t] = x[t];
    __syncthreads();

    // all threads in the block
    // now cooperate on tile[]
}
```

Block 0
private shared
tile[]

Block 1
separate shared
tile[]

Thread cooperation
within one block only

Synchronization

`__syncthreads()` is required when threads consume values produced by other threads in the same block.

```
extern __shared__ float dyn[];
kernel<<<B,T,T*sizeof(float)>>>(...);
```

Combined code walk-through

```
__device__ int d_var = 10;
__managed__ int m_var = 20;
__constant__ int c_var;

__device__ int square(int x) { return x*x; }

__global__ void kernel()
{
    __shared__ int s_var;
    if (threadIdx.x == 0) s_var = 5;
    __syncthreads();
    printf("%d %d %d", d_var, m_var, s_var);
}
```

__device__ variable
global GPU memory

__managed__ variable
CPU + GPU access

__constant__ variable
read-only kernel data

__device__ function
called by GPU code

__global__ function
kernel entry point

__shared__ variable
per-block scratchpad

Common mistakes and corrections

Mistake	Correction
Calling <code>__device__</code> function from CPU	Call it from a kernel, or make a <code>__host__ __device__</code> version.
Using <code>__shared__</code> across blocks	Use global memory or redesign communication; shared memory is per block.
Forgetting <code>cudaDeviceSynchronize()</code>	Synchronize before reading results or checking asynchronous errors.
Missing boundary check	Use <code>if (i < n)</code> before accessing arrays.
Assuming <code>__managed__</code> is always fastest	Profile; managed memory improves simplicity, not always speed.

Before compiling, mark each declaration as host function, device function, kernel, or memory-space variable.

Quick quiz

Q1

Which qualifier declares a CUDA kernel?

Answer: `__global__`

Q2

Which memory is shared only within a block?

Answer: `__shared__`

Q3

Which built-in identifies the current thread?

Answer: `threadIdx`

Q4

Which qualifier allows CPU and GPU access to one variable?

Answer: `__managed__`

Final summary

Function execution

- `__global__` = kernel launched with `<<<...>>>`
- `__device__` = GPU-side helper function
- `__host__` = CPU-side function
- `__host__ __device__` = two compiled versions

Variable placement

- `__device__` = GPU global variable
- `__managed__` = Unified Memory variable
- `__constant__` = read-only constant memory
- `__shared__` = per-block scratchpad

One-line rule

First ask where code executes; then ask where data lives.

CUDA C++ language extensions • execution configuration • Unified Memory